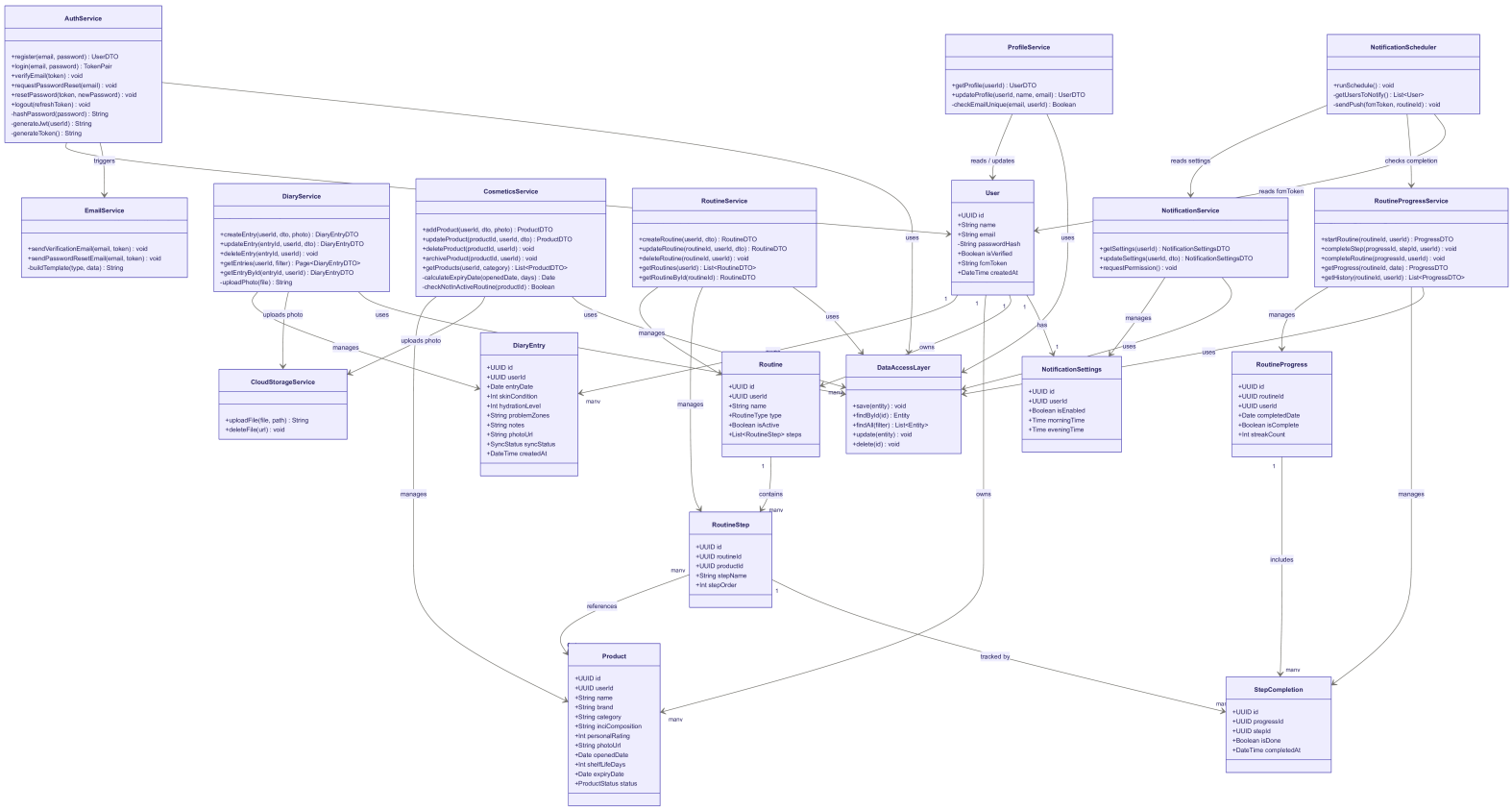
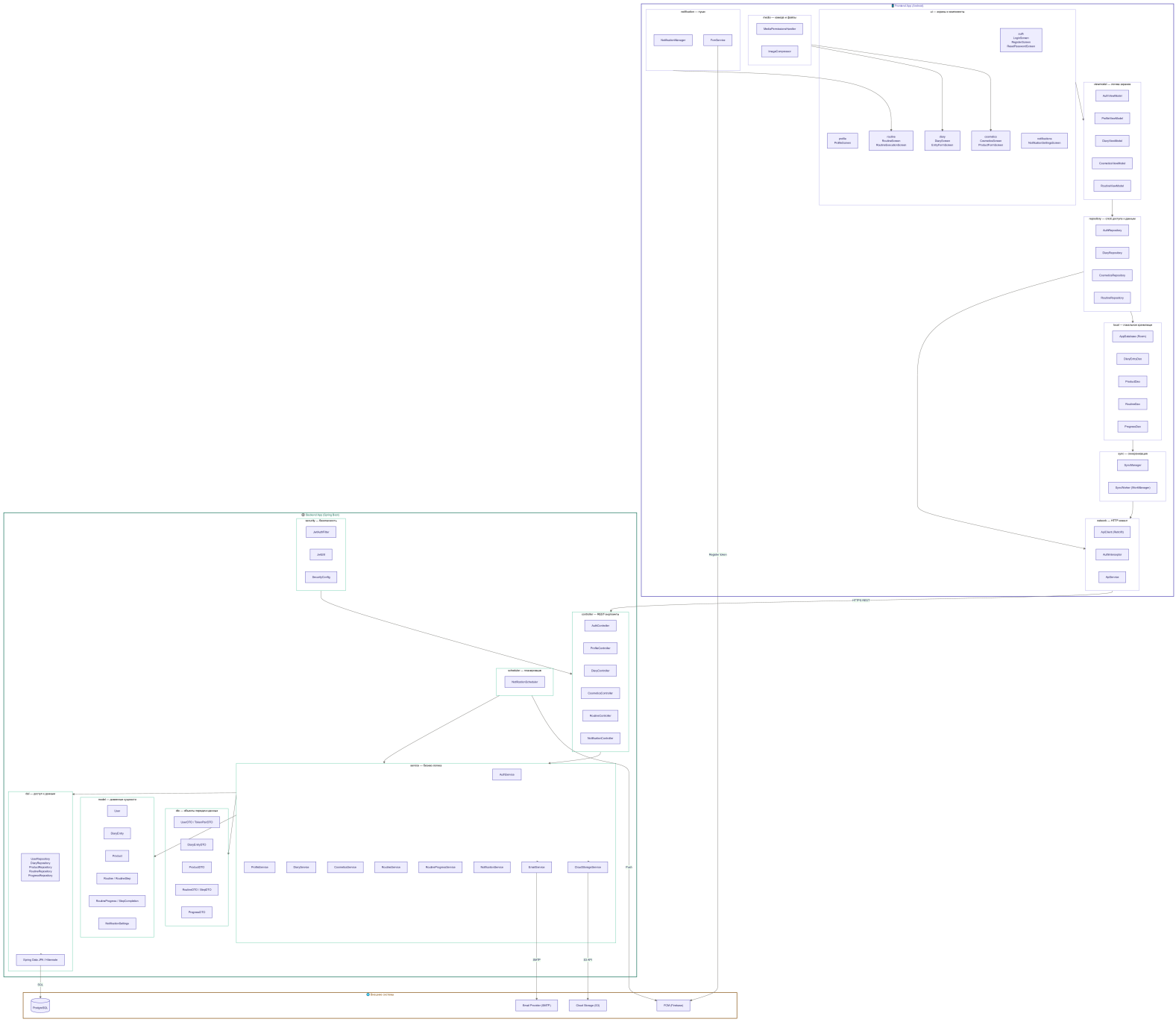
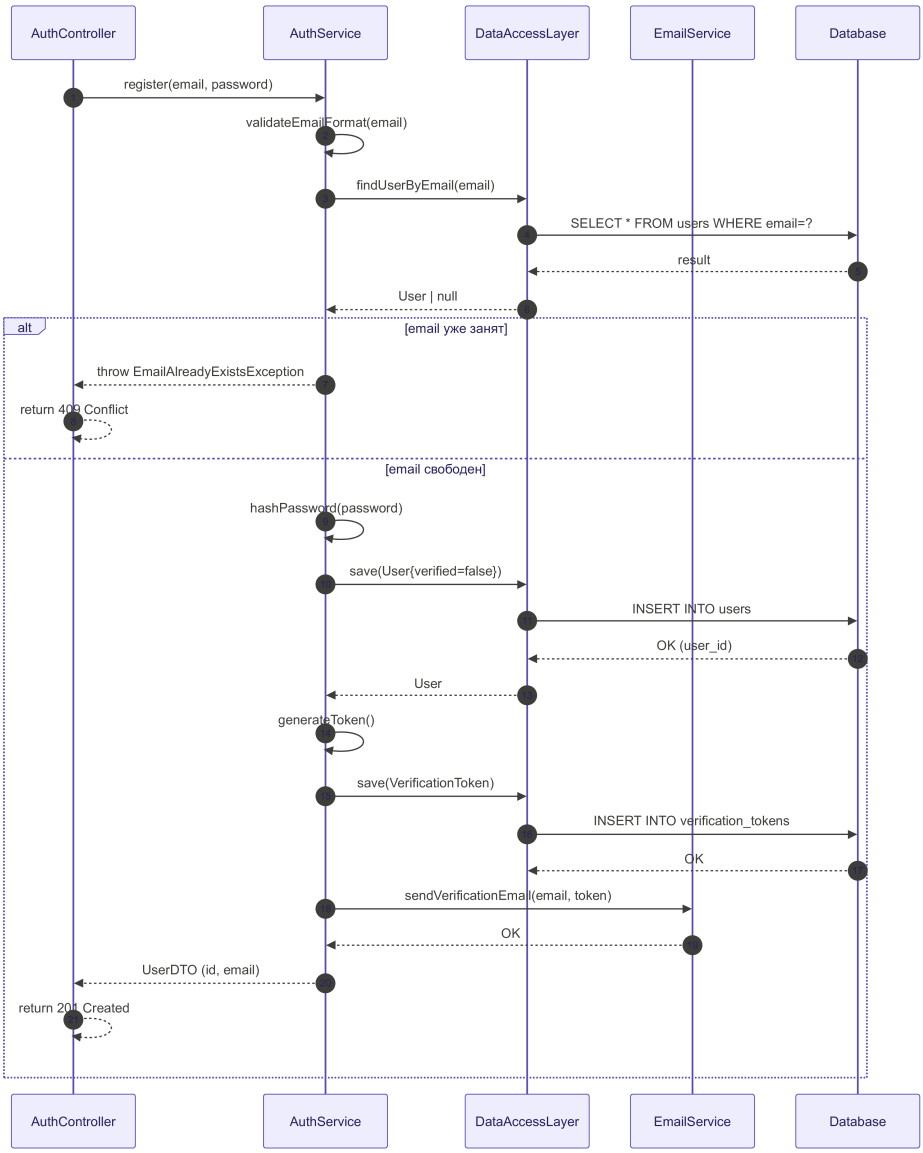
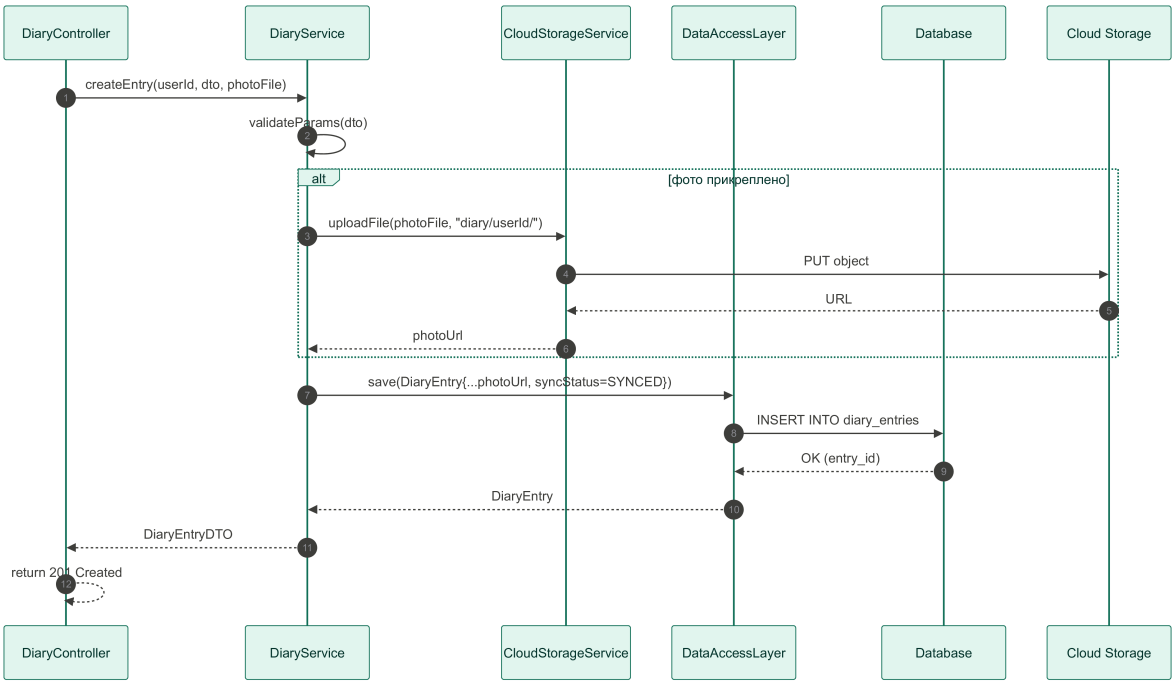
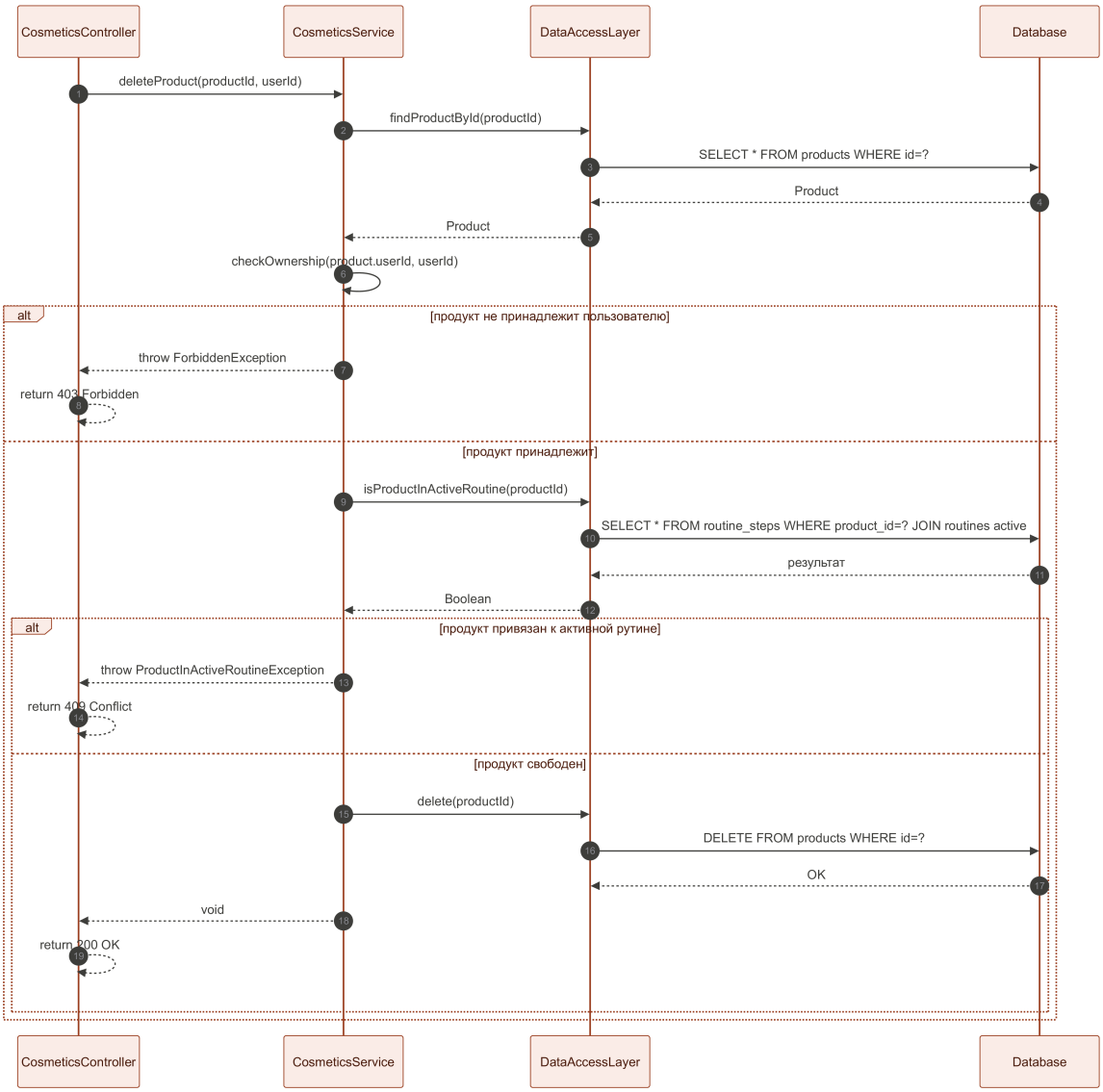
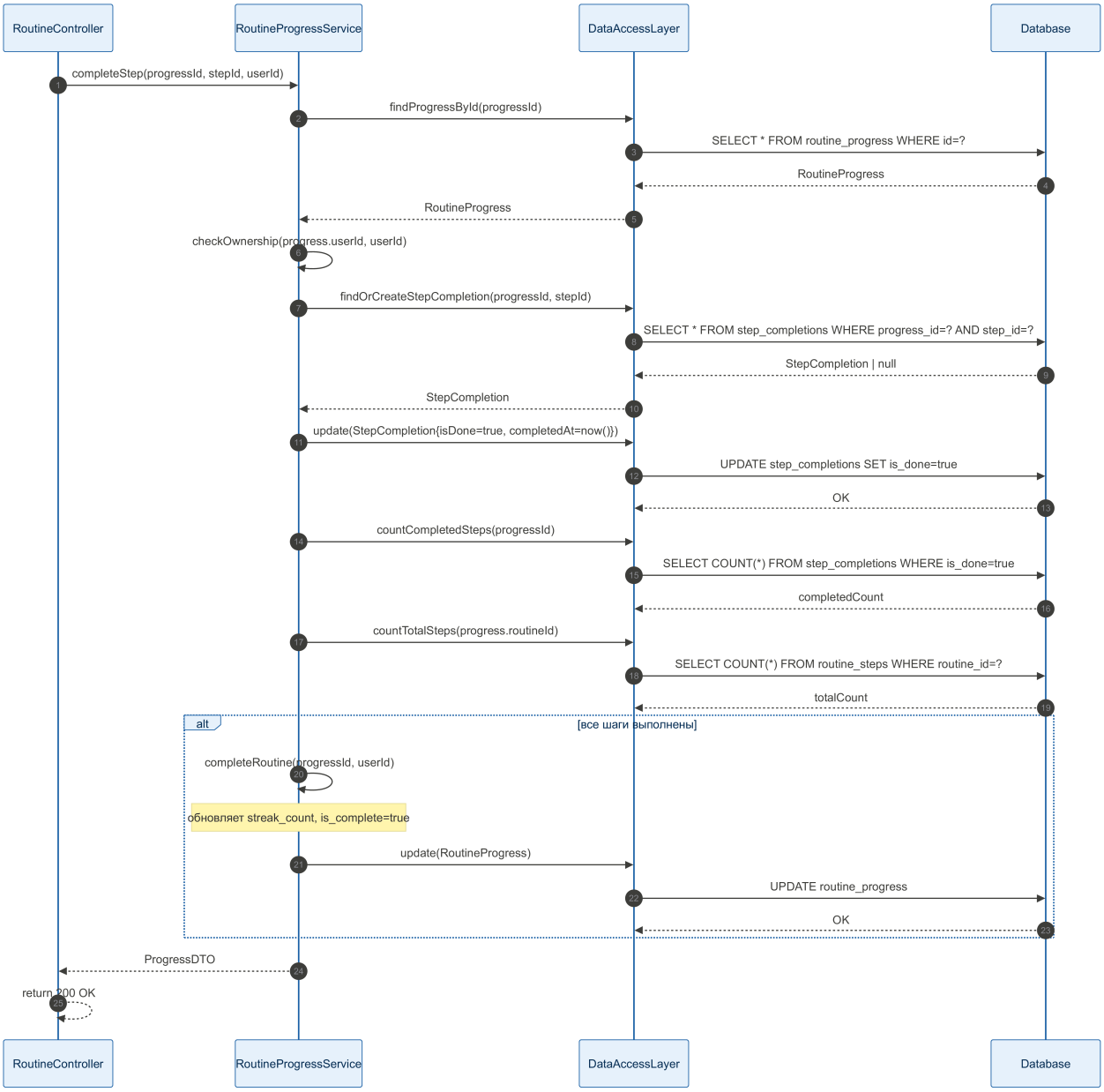

AuthService
+register(email, password) : UserDTO
+login(email, password) : TokenPair
+verifyEmail(token) : void
+requestPasswordReset(email) : void
+resetPassword(token, newPassword) : void
+logout(refreshToken) : void
-hashPassword(password) : String
-generateJwt(userId) : String
-generateToken() : String
ProfileService
+getProfile(userId) : UserDTO
+updateProfile(userId, name, email) : UserDTO
-checkEmailUnique(email, userId) : Boolean
NotificationScheduler
+runSchedule() : void
-getUsersToNotify() : List<User>
-sendPush(fcmToken, routineId) : void
EmailService
+sendVerificationEmail(email, token) : void
+sendPasswordResetEmail(email, token) : void
-buildTemplate(type, data) : String
DiaryService
+createEntry(userId, dto, photo) : DiaryEntryDTO
+updateEntry(entryId, userId, dto) : DiaryEntryDTO
+deleteEntry(entryId, userId) : void
+getEntries(userId, filter) : Page<DiaryEntryDTO>
+getEntryById(entryId, userId) : DiaryEntryDTO
-uploadPhoto(file) : String
CosmeticsService
+addProduct(userId, dto, photo) : ProductDTO
+updateProduct(productId, userId, dto) : ProductDTO
+deleteProduct(productId, userId) : void
+archiveProduct(productId, userId) : void
+getProducts(userId, category) : List<ProductDTO>
-calculateExpiryDate(openedDate, days) : Date
-checkNotInActiveRoutine(productId) : Boolean
RoutineService
+createRoutine(userId, dto) : RoutineDTO
+updateRoutine(routineId, userId, dto) : RoutineDTO
+deleteRoutine(routineId, userId) : void
+getRoutines(userId) : List<RoutineDTO>
+getRoutineById(routineId) : RoutineDTO
User
+UUID id
+String name
+String email
-String passwordHash
+Boolean isVerified
+String fcmToken
+DateTime createdAt
NotificationService
+getSettings(userId) : NotificationSettingsDTO
+updateSettings(userId, dto) : NotificationSettingsDTO
+requestPermission() : void
RoutineProgressService
+startRoutine(routineId, userId) : ProgressDTO
+completeStep(progressId, stepId, userId) : void
+completeRoutine(progressId, userId) : void
+getProgress(routineId, date) : ProgressDTO
+getHistory(routineId, userId) : List<ProgressDTO>
CloudStorageService
+uploadFile(file, path) : String
+deleteFile(url) : void
DiaryEntry
+UUID id
+UUID userId
+Date entryDate
+Int skinCondition
+Int hydrationLevel
+String problemZones
+String notes
+String photoUrl
+SyncStatus syncStatus
+DateTime createdAt
Routine
+UUID id
+UUID userId
+String name
+RoutineType type
+Boolean isActive
+List<RoutineStep> steps
DataAccessLayer
+save(entity) : void
+findById(id) : Entity
+findAll(filter) : List<Entity>
+update(entity) : void
+delete(id) : void
NotificationSettings
+UUID id
+UUID userId
+Boolean isEnabled
+Time morningTime
+Time eveningTime
RoutineProgress
+UUID id
+UUID routineId
+UUID userId
+Date completedDate
+Boolean isComplete
+Int streakCount
RoutineStep
+UUID id
+UUID routineId
+UUID productId
+String stepName
+Int stepOrder
Product
+UUID id
+UUID userId
+String name
+String brand
+String category
+String inciComposition
+Int personalRating
+String photoUrl
+Date openedDate
+Int shelfLifeDays
+Date expiryDate
+ProductStatus status
StepCompletion
+UUID id
+UUID progressId
+UUID stepId
+Boolean isDone
+DateTime completedAt
triggers
reads / updates
reads settings
checks completion
reads fcmToken
uses
uploads photo
manages
owns
has
contains
references
tracked by
includes
1
many

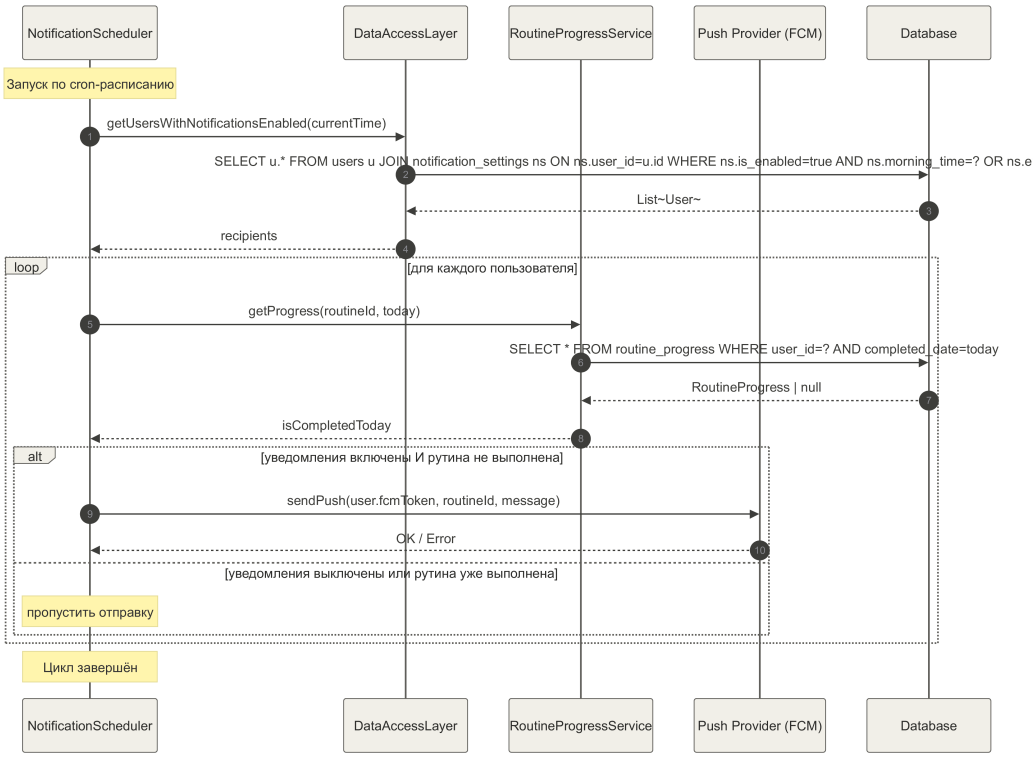

Frontend App (Android)
notification — пуши
NotificationManager
FcmService
media — камера и файлы
MediaPermissionsHandler
ImageCompressor
ui — экраны и компоненты
Auth
LoginScreen
RegisterScreen
ResetPasswordScreen
profile
ProfileScreen
routine
RoutineScreen
RoutineExecutorScreen
diary
DiaryScreen
EntryFormScreen
cosmetics
CosmeticsScreen
ProductFormScreen
notification
NotificationSettingsScreen
viewmodel — логика экранов
AuthViewModel
ProfileViewModel
DiaryViewModel
CosmeticsViewModel
RoutineViewModel
repository — слой доступа к данным
AuthRepository
DiaryRepository
CosmeticsRepository
RoutineRepository
local — локальная хранилище
AppDatabase (Room)
DiaryEntryDao
ProductDao
RoutineDao
ProgressDao
sync — синхронизация
SyncManager
SyncWorker (WorkManager)
network — HTTP-клиент
ApiClient (Retrofit)
AuthInterceptor
ApiService
Register token
HTTPS/REST
Backend App (Spring Boot)
security — безопасность
JwtAuthFilter
JwtUtil
SecurityConfig
controller — REST-контроллеры
AuthController
ProfileController
DiaryController
CosmeticsController
RoutineController
NotificationController
scheduler — планировщик
NotificationScheduler
service — бизнес-логика
AuthService
ProfileService
DiaryService
CosmeticsService
RoutineService
RoutineProgressService
NotificationService
EmailService
CloudStorageService
Push
SMTP
S3 API
dao — доступ к данным
UserRepository
DiaryRepository
ProductRepository
RoutineRepository
ProgressRepository
Spring Data JPA / Hibernate
model — доменные сущности
User
StepEntity
Product
Routine / RoutineStep
RoutineProgress / StepCompletion
NotificationSettings
dto — объекты передачи данных
UserDTO / TokenPair(DTO)
DiaryEntryDTO
ProductDTO
RoutineDTO / StepDTO
ProgressDTO
SQL
Внешние системы
PostgreSQL
Email Provider (SMTP)
Cloud Storage (S3)
FCM (Firebase)


AuthController
AuthService
DataAccessLayer
EmailService
Database
register(email, password)
validateEmailFormat(email)
findUserByEmail(email)
SELECT * FROM users WHERE email=?
result
User | null
alt
[email уже занят]
throw EmailAlreadyExistsException
return 409 Conflict
[email свободен]
hashPassword(password)
save(User{verified=false})
INSERT INTO users
OK (user_id)
User
generateToken()
save(VerificationToken)
INSERT INTO verification_tokens
OK
sendVerificationEmail(email, token)
OK
UserDTO (id, email)
return 201 Created
AuthController
AuthService
DataAccessLayer
EmailService
Database


DiaryController
DiaryService
CloudStorageService
DataAccessLayer
Database
Cloud Storage
1
createEntry(userId, dto, photoFile)
2
validateParams(dto)
alt
[фото прикреплено]
3
uploadFile(photoFile, "diary/userId/")
4
PUT object
5
URL
6
photoUrl
7
save(DiaryEntry{...photoUrl, syncStatus=SYNCED})
8
INSERT INTO diary_entries
9
OK (entry_id)
10
DiaryEntry
11
DiaryEntryDTO
12
return 201 Created
DiaryController
DiaryService
CloudStorageService
DataAccessLayer
Database
Cloud Storage


CosmeticsController
CosmeticsService
DataAccessLayer
Database
deleteProduct(productId, userId)
findProductById(productId)
SELECT * FROM products WHERE id=?
Product
Product
checkOwnership(product.userId, userId)
alt
[продукт не принадлежит пользователю]
throw ForbiddenException
return 403 Forbidden
[продукт принадлежит]
isProductInActiveRoutine(productId)
SELECT * FROM routine_steps WHERE product_id=? JOIN routines active
результат
Boolean
alt
[продукт привязан к активной рутине]
throw ProductInActiveRoutineException
return 409 Conflict
[продукт свободен]
delete(productId)
DELETE FROM products WHERE id=?
OK
void
return 200 OK
CosmeticsController
CosmeticsService
DataAccessLayer
Database


RoutineController
RoutineProgressService
DataAccessLayer
Database
1 completeStep(progressId, stepId, userId)
2 findProgressById(progressId)
3 SELECT * FROM routine_progress WHERE id=?
4 RoutineProgress
5 RoutineProgress
6 checkOwnership(progress.userId, userId)
7 findOrCreateStepCompletion(progressId, stepId)
8 SELECT * FROM step_completions WHERE progress_id=? AND step_id=?
9 StepCompletion | null
10 StepCompletion
11 update(StepCompletion{isDone=true, completedAt=now()})
12 UPDATE step_completions SET is_done=true
13 OK
14 countCompletedSteps(progressId)
15 SELECT COUNT(*) FROM step_completions WHERE is_done=true
16 completedCount
17 countTotalSteps(progress.routineId)
18 SELECT COUNT(*) FROM routine_steps WHERE routine_id=?
19 totalCount
alt
[все шаги выполнены]
20 completeRoutine(progressId, userId)
обновляет streak_count, is_complete=true
21 update(RoutineProgress)
22 UPDATE routine_progress
23 OK
24 ProgressDTO
25 return 200 OK
RoutineController
RoutineProgressService
DataAccessLayer
Database


NotificationScheduler
DataAccessLayer
RoutineProgressService
Push Provider (FCM)
Database
Запуск по cron-расписанию
1
getUsersWithNotificationsEnabled(currentTime)
2
SELECT u.* FROM users u JOIN notification_settings ns ON ns.user_id=u.id WHERE ns.is_enabled=true AND ns.morning_time=? OR ns.e
3
List~User~
4
recipients
loop
[для каждого пользователя]
5
getProgress(routineId, today)
6
SELECT * FROM routine_progress WHERE user_id=? AND completed_date=today
7
RoutineProgress | null
8
isCompletedToday
alt
[уведомления включены И рутина не выполнена]
9
sendPush(user.fcmToken, routineId, message)
10
OK / Error
[уведомления выключены или рутина уже выполнена]
пропустить отправку
Цикл завершён
NotificationScheduler
DataAccessLayer
RoutineProgressService
Push Provider (FCM)
Database